

DataOps Mastery: Advanced CI/CD and Pipeline Automation

Module 4 – Build, deploy, run, iterate

Asst. Prof. Dr. Santitham Prom-on

Department of Computer Engineering, Faculty of Engineering
King Mongkut's University of Technology Thonburi

Course Overview

DAY 1

FOUNDATIONS OF GIT, YAML AND DATABRICKS



MODULE 1

Git that survives a real team

- CI/CD concepts: why notebook-driven delivery breaks
- Introduction to Git, version control in data work
- add, commit, push, pull
- Branching, merging, conflicts
- Databricks Repos: Git inside the workspace



MODULE 2

YAML and the Databricks workspace

- YAML, deep-dive- syntax, context, mappings, sequences
- The traps that break CI; anchors and reuse
- Databricks workspace (re)overview
- Cluster, serverless and job compute
- Databricks CLI commands, operations and authentication



DAY 2

DATABRICKS ASSET BUNDLE



MODULE 3

Bundle anatomy and your first deployment

- What problem do bundles solve?
- Anatomy of databricks.yml
- Variables, targets and development mode
- Initialize and explore a bundle



MODULE 4

Build, deploy, run, iterate

- Defining jobs, tasks and clusters in YAML
- Multi-task pipelines and dependencies
- Targets and variable overrides
- The validate to deploy to run loop
- Debugging deploys: failure modes and a diagnosis recipe



DAY 3

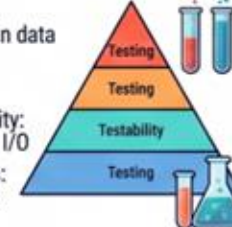
TESTING AND CONTINUOUS INTEGRATION



MODULE 5

Testing pipelines: writing code you can verify

- What's worth testing in data and ML work
- The testing pyramid
- Designing for testability: separating logic from I/O
- Writing the test cases: pytest, fixtures, Spark assertions



MODULE 5

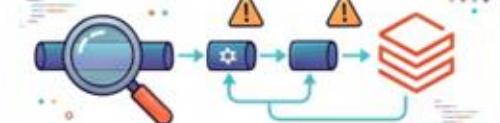
CI/CD principles: workflows, jobs, steps

- GitHub Actions architecture and composition
- Workflows, jobs, steps, runners and triggers
- Marketplace actions
- Securing workflows, secrets and masking



DAY 4

E2D-TO-END CI/CD ON DATABRICKS



MODULE 6

Wiring GitHub Actions to Databricks

- Authentication: service principals and OAuth
- Environments, environment secrets and approval gates
- Workflow pattern: validate on PR, deploy on merge
- Multi-environment promotion; reusable workflows in practice



MODULE 8

Production-readiness and capstone

- Integration and smoke tests against a deployed bundle
- Monitoring and alerting
- Rollback and troubleshooting patterns
- Capstone



Module overview

- Author a job resource from written requirements: tasks, payloads and compute.
- Declare a job cluster, bind tasks to it, and say what each field controls.
- Express a dependency graph, and predict each task's state when one fails.
- Import an existing workspace job and correct the defects it arrives with.
- Deploy one bundle to two targets, overriding only what differs.
- Diagnose a failed deployment by identifying the stage that reported it.

Declaring a job

Part 1

The starting point

bundle init default-minimal — a bundle with nowhere for a job to go yet

```
$ databricks bundle init default-minimal
$ tree -a -I '.databricks|.git'
.
|-- databricks.yml          bundle, include, variables, dev and prod targets
|-- pyproject.toml          package metadata
|-- tests/conftest.py
|-- fixtures/.gitkeep
|-- AGENTS.md  CLAUDE.md  README.md
`-- .vscode/

$ databricks bundle validate
Validation OK!
```

databricks.yml declares include: [resources/*.yml], and the directory does not exist. A glob that matches nothing is not an error. Everything you write today goes into that directory.

```
santitham@data-engineer:~/m4-demo$ databricks bundle init default-minimal
Welcome to the minimal Declarative Automation Bundle template!
```

This template creates a minimal project structure without sample code, ideal for advanced users.
(For getting started with Python or SQL code, use the default-python or default-sql templates instead.)

Your workspace at <https://adb-1502583690645883.3.azuredatabricks.net> is used for initialization.
(See <https://docs.databricks.com/dev-tools/cli/profiles.html> for how to change your profile.)

Unique name for this project [my_project]: minimal_project
Default catalog for any tables created by this project [ctl_training_dev]:
Initial language for this project: python

✨ Your new project has been created in the 'minimal_project' directory!

To get started, refer to the project README.md file and the documentation at <https://docs.databricks.com/dev-tools/bundles/index.html>.

```
santitham@data-engineer:~/m4-demo/minimal_project$ tree -a -I '.databricks|.git'
```

```
.
├── .gitignore
├── .vscode
│   ├── __builtins__.pyi
│   ├── extensions.json
│   └── settings.json
├── AGENTS.md
├── CLAUDE.md
├── README.md
├── databricks.yml
├── fixtures
│   └── .gitkeep
├── pyproject.toml
├── resources
├── src
├── tests
│   └── conftest.py
```

```
santitham@data-engineer:~/m4-demo/minimal_project$ databricks bundle validate
Name: minimal_project
Target: dev
Workspace:
  Host: https://adb-1502583690645883.3.azuredatabricks.net
  User: santitham.pro@kmutt.ac.th
  Path: /Workspace/Users/santitham.pro@kmutt.ac.th/.bundle/minimal_project/dev

Validation OK!
```

The smallest job

Five lines. Every slide in Parts A and B adds or replaces exactly one of them.

```
# resources/ingest.job.yml
resources:
  jobs:
    ingest:                                <-- the key you chose
      name: sales_ingest                  <-- what the workspace will display
      tasks:
        - task_key: land
          notebook_task:
            notebook_path: ../src/land.py
```

This validates and deploys. It creates a job with one task, and the task has no compute declared, which Part B fixes.

task_key is an identity, not a label

What refers to it, and what does not check it

- **Three things refer to a task by its key**
depends_on. bundle run --only, when you rerun one task of a deployed job. And the run history, which reports state per key.
- **Two tasks may carry the same key without complaint**
bundle validate reports Validation OK! for a job with two tasks keyed clean. The Jobs API rejects it at deploy. Uniqueness is enforced there, not locally.

One payload key per task

The key that names the work. A task declares exactly one of these.

Payload key	Runs	Names
<code>notebook_task</code>	a notebook in the workspace	<code>notebook_path</code> , <code>base_parameters</code>
<code>spark_python_task</code>	a Python file, with Spark available	<code>python_file</code> , <code>parameters</code>
<code>python_wheel_task</code>	an entry point in a built wheel	<code>package_name</code> , <code>entry_point</code>
<code>sql_task</code>	a query or file on a SQL warehouse	<code>warehouse_id</code> , then query or file
<code>run_job_task</code>	another job	<code>job_id</code>
<code>condition_task</code>	a comparison that gates what follows	<code>op</code> , <code>left</code> , <code>right</code>

This course uses the first two. `python_wheel_task` is what the default-python template you saw this morning declares, which is why that template needs `uv`.

notebook_task

The path is relative to the file that declares it, not to the bundle root

```
# resources/ingest.job.yml           the file is in resources/  
tasks:  
  - task_key: land  
    notebook_task:  
      notebook_path: ../src/land.py  
      base_parameters:  
        catalog: ${var.catalog}  
        schema: ${var.schema}
```

base_parameters are read inside the notebook with `dbutils.widgets.get`. This is how a notebook receives the catalog for the target it is running in, rather than hard-coding one.

spark_python_task

The same task, with the payload replaced

```
tasks:  
  - task_key: land  
    spark_python_task:  
      python_file: ../src/land.py  
      parameters:  
        - --catalog  
        - ${var.catalog}  
        - --lookback-days  
        - "7"
```

parameters is a sequence of strings, passed to the file as command-line arguments and read with argparse. Note the quotes on "7": Module 2's YAML rules still apply, and an unquoted 7 is an integer where a string is required.

A local path becomes a workspace path

Resolved before any deployment, and the rewriting differs by payload key

Declared in resources/ingest.job.yml	Resolved for target dev
<pre>notebook_task: notebook_path: ../src/land.py</pre>	<pre>.../.bundle/m4demo/dev/files/src/land</pre>
<pre>spark_python_task: python_file: ../src/land.py</pre>	<pre>.../.bundle/m4demo/dev/files/src/land.py</pre>

The notebook loses its extension because a notebook is a workspace object, not a file. The Python file keeps it because it stays a file. Read either value with `bundle validate -o json` before you deploy.

What makes a .py file a notebook

The one rule in Part A enforced by reading the file rather than its path

- **The first line must be the notebook marker**

Databricks notebook source

Nothing else distinguishes a notebook from a Python file on disk. The marker is what the workspace import path looks for.

- **Without it, the same declaration is rejected**

Error: expected a notebook for "resources.jobs.ingest.tasks[1]
.notebook_task.notebook_path" but got a file: file at .../src/land.py
is not a notebook

A file you wrote by hand will not have the marker unless you add it. A file exported from the workspace will. This is the most common failure in the next hour.

Keys that belong on the job, not on a task

Key	Controls	Note
<code>max_concurrent_runs</code>	how many runs may overlap	development mode sets 4
<code>timeout_seconds</code>	the whole job's limit, 0 for none	tasks have their own
<code>tags</code>	key/value labels in the workspace	development mode adds one
<code>email_notifications</code>	<code>on_start</code> , <code>on_success</code> , <code>on_failure</code>	each is a list of addresses

Compute for a task

Part 2

Three ways a task obtains compute

The declaration, and the consequence of each

Declared on the task	Compute	Started
<code>job_cluster_key: main</code>	a cluster this job creates	at run time, terminated after
<code>existing_cluster_id: 0815-...</code>	an interactive cluster already there	it is already running
<code>environment_key: default</code>	serverless	managed by the platform

`existing_cluster_id` names one cluster in one workspace, so a bundle carrying it cannot be promoted. That is the reason this course declares job clusters.

job_clusters is declared on the job

A sequence. Each entry requires exactly two keys.

```
resources:
  jobs:
    ingest:
      name: sales_ingest
      job_clusters:
        - job_cluster_key: main      <-- the identity of this entry
          new_cluster:              <-- the specification
            spark_version: 16.4.x-scala2.12
            node_type_id: Standard_D3_v2
            data_security_mode: DATA_SECURITY_MODE_AUTO
            autoscale:
              min_workers: 1
              max_workers: 4
      tasks:
        - task_key: land
          job_cluster_key: main      <-- the reference to it
          notebook_task:
            notebook_path: ../src/land.py
```

job_cluster_key appears twice

Once as an identity, once for each task that claims it

- **On the job_clusters entry, it is the name of the specification**
Nothing else in the file gives that specification a handle.
- **On a task, it is a claim on that specification**
Two tasks naming the same key run on one cluster, one after the other. Two tasks naming different keys run on two clusters, and may run at the same time.
- **A cluster no task claims is still created**
It appears in the deployed job definition and starts nothing. Part E shows the override mistake that produces one.

The fields of `new_cluster`

Four fields fix everything this course depends on

Field	Value used here	Fixes
<code>spark_version</code>	<code>16.4.x-scala2.12</code>	the runtime, and so Spark and Python
<code>node_type_id</code>	<code>Standard_D3_v2</code>	the instance type of driver and workers
<code>data_security_mode</code>	<code>DATA_SECURITY_MODE_AUTO</code>	the access mode, and what UC permits
<code>autoscale</code> / <code>num_workers</code>	see the next slide	the worker count

16.4 is the long-term support runtime the Databricks templates currently pin, supported to May 2028. Documentation examples still showing 13.3.x-scala2.12 name a runtime that goes out of support in August 2026.

num_workers or autoscale, not both

Two ways to state the worker count

A FIXED COUNT

```
num_workers: 2
```

Two workers for the whole run.
Predictable cost. Predictable
duration. Nothing adapts if the
input is larger than usual.

A RANGE

```
autoscale:
```

```
  min_workers: 1
```

```
  max_workers: 4
```

Starts at one and adds workers
while there is queued work.
max_workers must exceed
min_workers.

Declaring both passes validation and resolves with both present. The conflict is not reported until the Jobs API receives it at deploy.

One cluster for the job, or one for each task

The same three tasks, declared two ways

ONE ENTRY, THREE TASKS CLAIM IT

One cluster start, once, for the whole run.

The three tasks run in sequence on shared compute even where the graph would allow two of them in parallel.

One size has to suit all three.

Cheapest, and slowest.

THREE ENTRIES, ONE EACH

Three cluster starts. Each is a few minutes of the run.

Tasks the graph allows in parallel actually run in parallel.

Each task is sized for its own work: the features task gets the workers, the notify task does not.

The choice is between start-up cost and per-task sizing, and it is made in the file. Nothing outside the declaration adjusts it.

What validate reports when the binding is wrong

A task claims mian; the entry is keyed main

YOU RUN

```
$ databricks bundle validate

# resources/ingest.job.yml, extract
  job_clusters:
    - job_cluster_key: main
      new_cluster:
        ...
  tasks:
    - task_key: land
      job_cluster_key: mian
        ^ one transposition
```

YOU SEE

```
Warning: job_cluster_key mian is
not defined
  at resources.jobs.ingest
    .tasks[1].job_cluster_key
  in resources/ingest.job.yml:42:28

Name: m4demo
Target: dev
...

Found 1 warning
```

Read the whole message, not the last line

The warning above carries three separate pieces of information

- **It is a warning, not an error**
The command reports Found 1 warning and exits successfully. A participant who reads only the last line will deploy a job whose task has no compute.
- **at resources.jobs.ingest.tasks[1].job_cluster_key**
The configuration path: the address in the resolved configuration, which is what you query with bundle validate -o json.
- **in resources/ingest.job.yml:42:28**
The file, the line and the column. This is where you edit.

Challenge 1 — Author a job on a job cluster

- 1 Read assets/challenge1-requirements.md. Do not open any existing resource file.
- 2 Write resources/ingest.job.yml from the requirements: one job, one notebook task, one job cluster that you declare.
- 3 Create the notebook the task names, with the marker line as its first line.
- 4 Before you validate, write down the workspace path you expect notebook_path to resolve to.
- 5 Run databricks bundle validate, then read the resolved path out of the JSON output and compare it with your prediction.

Success: Validation OK!, and a resolved notebook_path matching what you wrote down.

Importing a job that already exists

Part 3

bundle generate job

One command reads a job out of the workspace and writes it as bundle source

- **databricks bundle generate job --existing-job-id <id> --key <key>**
Reads the job through the Jobs API, writes a resource file, and downloads every notebook the job names.
- **Two destinations, both with defaults**
--config-dir, default resources/, receives the YAML. --source-dir, default src/, receives the notebooks. --key sets the resource key; without it the CLI picks one.
- **Only notebook tasks are supported**
A job whose tasks are wheel or SQL tasks is not generated. Add --download-spark-python-files to bring down the files a spark_python_task names.

```
santitham@data-engineer:~/m4-demo$ cd minimal_project/  
santitham@data-engineer:~/m4-demo/minimal_project$ databricks bundle generate job --existing-job-id 752310395422527  
File successfully saved to src/hello_world.py  
Job configuration successfully saved to resources/hello_world_santitham.job.yml
```

What it produces

Generated from a one-task job built in the interface on an interactive cluster

```
resources:
  jobs:
    hello_world:
      name: hello_world_ui
      tasks:
        - task_key: hello
          existing_cluster_id: 0815-123456-abcdefgh
          email_notifications: {}
          notebook_task:
            notebook_path: ../src/hello_world.py
            source: WORKSPACE
            run_if: ALL_SUCCESS
            timeout_seconds: 0
            webhook_notifications: {}
          email_notifications: {}
          max_concurrent_runs: 1
          queue:
            enabled: true
          timeout_seconds: 0
          webhook_notifications: {}
```

```
santitham@data-engineer:~/m4-demo/minimal_project$ yq resources/hello_world_santitham.job.yml
resources:
  jobs:
    hello_world_santitham:
      name: hello-world-santitham
      tasks:
        - task_key: say-hello
          existing_cluster_id: 0318-031919-b3fa4xtr
          disabled: false
          email_notifications: {}
          notebook_task:
            notebook_path: ../src/hello_world.py
            source: WORKSPACE
            run_if: ALL_SUCCESS
            timeout_seconds: 0
          email_notifications: {}
          max_concurrent_runs: 1
          queue:
            enabled: true
          timeout_seconds: 0
          webhook_notifications: {}
```

Three defects, and what reports them

Every one of these passes bundle validate

In the generated file	Why it is wrong	Reported by
<code>existing_cluster_id: 0815-...</code>	names a cluster in one workspace; no other target has it	nothing, until you deploy to a second target
<code>source: WORKSPACE</code>	the file is bundle source now, synchronised with the bundle	nothing
<code>nine keys carrying defaults or empty mappings</code>	no information, and they hide the nine lines that carry some	nothing

The generated file is a starting point, not an output. Reading it is the fastest way to learn the shape of a job, which is why this part follows Parts 1 and 2 rather than replacing them.

Challenge 2 — Port a job and repair it

- 1 Take the job id from the instructor and run bundle generate job with --key ported.
- 2 Read the generated file and mark the three defects from the previous slide before you change anything.
- 3 Replace existing_cluster_id with a job cluster you declare, remove the keys that carry only defaults, and drop source: WORKSPACE.
- 4 Validate, then confirm from the JSON output that the resolved notebook_path is inside the bundle's files root rather than your home directory.

Success: A resource file of under fifteen lines, all of them carrying information, that validates.

Multi-task jobs

Part 4

depends_on

A sequence of mappings, each naming one task_key

- **A task with depends_on does not start until its dependencies are terminal**
Terminal means finished, in any state. Whether the task then runs is decided by run_if, three slides from here.
- **A task without depends_on starts when the run starts**
Several such tasks start together. This is where parallelism comes from; nothing else in the file requests it.
- **The dependency is declared downstream**
land does not say what follows it; features says what it waits for. The order of entries in the tasks sequence therefore carries no meaning at all.

A three-task chain

land, then features, then publish — one cluster, claimed three times

```
tasks:
  - task_key: land
    job_cluster_key: main
    notebook_task:
      notebook_path: ../src/land.py

  - task_key: features
    depends_on:
      - task_key: land
    job_cluster_key: main
    notebook_task:
      notebook_path: ../src/features.py

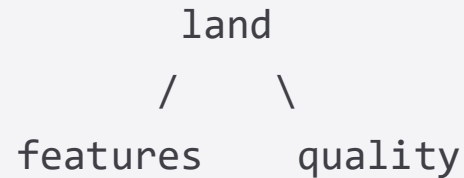
  - task_key: publish
    depends_on:
      - task_key: features
    job_cluster_key: main
    notebook_task:
      notebook_path: ../src/publish.py
```

The graph a declaration produces

Three shapes, and the depends_on that produces each

FAN-OUT: TWO TASKS, ONE PARENT

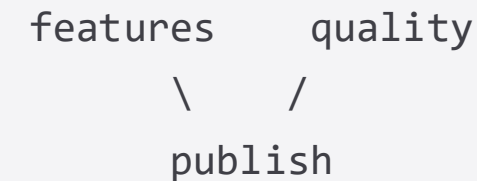
```
features: depends_on: [land]
quality:  depends_on: [land]
```



Both start when land succeeds, and run at the same time if each has its own cluster.

FAN-IN: ONE TASK, TWO PARENTS

```
publish:
  depends_on:
    - task_key: features
    - task_key: quality
```



publish waits for both.

A chain is the degenerate case of both. Nothing in the file draws the graph; the graph is what the set of depends_on entries implies.

run_if

Six values, and the CLI prints all six when you give it a seventh

Value	The task runs when its dependencies	Used for
ALL_SUCCESS	all succeeded — the default	the ordinary case
AT_LEAST_ONE_SUCCESS	at least one succeeded	partial-input work
NONE_FAILED	none failed, skipped or not	optional upstream steps
ALL_DONE	all reached a terminal state	cleanup, notification
AT_LEAST_ONE_FAILED	at least one failed	compensating work
ALL_FAILED	all failed	a fallback path

ALL_DONE is what a notification task declares, so that it runs whether or not the pipeline succeeded.
ALL_DONE_OR_SKIPPED and NONE_FAILED_OR_SKIPPED do not exist.

What a failure does to the tasks below it

The states a run reports, and which of them the code produced

State	Meaning	Produced by
SUCCESS	the task finished	the code
FAILED	the task raised, or exited non-zero	the code
TIMEDOUT	the task exceeded timeout_seconds	the code, or the cluster
UPSTREAM_FAILED	a dependency failed, so this never ran	the platform
SKIPPED	run_if was not satisfied	the platform
CANCELED	the run was stopped	a person, or a concurrency limit

A downstream task of a failed task is not reported as a failure. The run history distinguishes the task that broke from the tasks that were prevented, which is what lets you find the first cause rather than the last symptom.

Retries

Declared per task, and they re-run only that task

- **max_retries**
How many further attempts after the first failure. -1 means retry indefinitely, which is a decision, not a default.
- **min_retry_interval_millis**
The wait between the end of one attempt and the start of the next. Without it, a task that fails on a rate limit retries straight into the same rate limit.
- **retry_on_timeout**
Whether a TIMEDOUT attempt counts as retrieable. False by default, because a task that ran out of time will usually do so again.

A retry re-runs the task on the same cluster and does not re-run anything above it. A task that is not idempotent must not be given retries.

Three graph faults that validate accepts

All three return Validation OK!, and all three are rejected at deploy

- **depends_on naming a task that does not exist**
A transposed key in a dependency reads as a dependency on nothing.
- **A cycle**
land depends on publish, publish depends on features, features depends on land.
No task can start.
- **Two tasks sharing a task_key**
The second is not a second task; the definition is ambiguous.

Validation checks the schema and resolves the references it owns. It does not evaluate the graph, so the resource stage of deploy is the first place these appear. That is the distinction the recipe in Part 6 is built on.

Challenge 3 — Build the pipeline

- 1 Read assets/challenge3-requirements.md and draw the graph it describes before writing anything.
- 2 Declare four tasks: one root, two that fan out from it, and one that fans in from both.
- 3 Give the fan-out tasks their own job clusters so they can actually run in parallel, and the others the shared one.
- 4 Add a fifth task that notifies, with `run_if: ALL_DONE` and no cluster of its own.
- 5 Give one retry to the task most likely to fail, with an interval, and say in one line why that task and not another.
- 6 Validate, then check your drawn graph against the `depends_on` entries you wrote.

Success: Validation OK!, and a drawing that matches the file rather than your intention.

Targets and overrides

Part 5

What a second environment differs by

Four properties. A target block that contains anything else is doing too much.

Property	Expressed by	dev to staging
the workspace	<code>targets.<t>.workspace.host</code>	may be the same host
where data is written	a variable, per target	catalog and schema differ
the size of the compute	a cluster override, or a complex variable	1-4 workers to 2-8
whether schedules fire	mode, or <code>trigger.pause_status</code>	paused to unpaused

Everything else is identical, and identical is the point. A target that overrides the task graph is two pipelines maintained in one file.

Adding a staging target

mode: production, an explicit root path, and the two variables reassigned

```
targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: training
      schema: ${workspace.current_user.short_name}

  staging:
    mode: production
    workspace:
      root_path: /Workspace/Users/${workspace.current_user.userName}/\
                .bundle/${bundle.name}/${bundle.target}
    variables:
      catalog: training
      schema: staging
    permissions:
      - user_name: ${workspace.current_user.userName}
        level: CAN_MANAGE
```

targets.<t>.resources merges into the base

A target may carry a partial resource definition

- **It is merged, not substituted**

The base declaration is read first. The target's fields are then laid over it, field by field, at every level of nesting.

- **So a target block names only what differs**

Not the job. Not the task list. Not the fields of the cluster that are the same in both. Only the ones that change.

- **And the base file is never edited to add a target**

resources/ingest.job.yml is written once, in Part D, and stays as it is for the rest of the course.

The merge, resolved

Staging declares one key. Read what the CLI resolves.

YOU RUN

```
# databricks.yml
targets:
  staging:
    resources:
      jobs:
        ingest:
          job_clusters:
            - job_cluster_key: main
              new_cluster:
                autoscale:
                  min_workers: 2
                  max_workers: 6

$ databricks bundle validate \
  -t staging -o json | jq \
  '.resources.jobs.ingest
  .job_clusters[0]'
```

YOU SEE

```
{
  "job_cluster_key": "main",
  "new_cluster": {
    "autoscale": {
      "max_workers": 6,
      "min_workers": 2
    },
    "data_security_mode":
      "DATA_SECURITY_MODE_AUTO",
    "node_type_id":
      "Standard_D3_v2",
    "spark_version":
      "16.4.x-scala2.12"
  }
}
```

three fields from the base,
one from the target

Sequences merge by key, and a typo appends

The same override with mian instead of main

YOU RUN

```
# databricks.yml
targets:
  staging:
    resources:
      jobs:
        ingest:
          job_clusters:
            - job_cluster_key: mian
              new_cluster:
                autoscale:
                  min_workers: 2
                  max_workers: 6

$ databricks bundle validate \
  -t staging

Validation OK!
```

YOU SEE

job_clusters now has FOUR entries, not three:

main	autoscale 1-3, untouched, and still claimed by land, publish and notify
features	4 workers
quality	2 workers
mian	autoscale 2-6 and nothing else: no runtime, no node type, claimed by no task

No warning. No error. Staging deploys, runs on 1-3 workers, and creates a cluster that nothing uses.

Why that override was silent

The merge matched on a key, and the key it was given did not exist

- **Sequence entries are matched by their key field**
job_clusters on job_cluster_key, tasks on task_key. A matching key merges into the entry. A key that matches nothing is a new entry, which is a legitimate thing for a target to add.
- **So the CLI cannot tell your typo from your intention**
Adding a staging-only task is the same operation as misspelling an existing one. There is no warning available to give.
- **The check is to read the resolved configuration, not the file**
bundle validate -o json for the target, and count the entries. This is the one fault in the module that no message will ever point at.

Where a variable's value comes from

Five sources, lowest precedence first. Measured, not quoted.

#	Source	Set by
1	default: in the variable declaration	the author of the bundle
2	the selected target's variables: block	the author, per environment
3	<code>.databricks/bundle/<target>/variable-overrides.json</code>	a developer, locally
4	the <code>BUNDLE_VAR_<name></code> environment variable	a shell, or a CI runner
5	<code>--var name=value</code> on the command line	whoever typed the command

A variable may hold a whole mapping

type: complex, and the job references it as one value

```
variables:
  cluster:
    description: The job cluster specification for this target
    type: complex
    default:
      spark_version: 16.4.x-scala2.12
      node_type_id: Standard_D3_v2
      data_security_mode: DATA_SECURITY_MODE_AUTO
      autoscale: { min_workers: 1, max_workers: 4 }

# resources/ingest.job.yml
  job_clusters:
    - job_cluster_key: main
      new_cluster: ${var.cluster}

# databricks.yml, in the staging target
variables:
  cluster:
    spark_version: 16.4.x-scala2.12
    node_type_id: Standard_D3_v2
    num_workers: 8
```

Merge or replace

The two mechanisms behave differently, so the choice is deliberate

Mechanism	A partial value	Choose it when
<code>targets.<t>.resources</code>	merges field by field into the base	one or two fields differ, and the rest must stay in one place
<code>a complex variable</code>	replaces the whole value	the specification differs as a unit, and a partial copy would mislead

The staging assignment on the previous slide carries `num_workers` and no `autoscale`, and the resolved cluster has `num_workers` and no `autoscale`. Nothing was inherited. That is the whole difference between the two rows.

Challenge 4 — Promote to staging

- 1 Add a staging target that differs from dev in the catalog, the worker count and whether the schedule fires. Do not edit `resources/ingest.job.yml`.
- 2 Resolve both targets to JSON and diff them. Account for every line that differs, including the ones you did not write.
- 3 Count the entries in `job_clusters` for staging. Say why counting is necessary here and nowhere else.
- 4 For each of the two variables, name which of the five sources supplied its value in staging, and which supplied it in dev.

Success: Two resolutions differing in exactly the properties you declared, and no extra cluster.

The loop, and the diagnosis

Part 5

The five steps, and what each one can tell you

Step	Reads	Can report
edit	nothing	your editor's YAML errors, and no more
validate	the files, and the workspace	schema faults, unresolved references
plan	the above, and deployment state	what would be created, changed, deleted
deploy	the above, and the Jobs API	build, upload, resource and state faults
run	the deployed job	per-task states, and your code's failures

The step that reported a failure bounds the set of possible causes before you read anything else. That is the first question in the recipe two slides from here.

plan before deploy

The action the deployment would take, per resource

```
$ databricks bundle plan -t dev
```

```
santitham@data-engineer:~/m4-demo/assets/m4demo$ databricks bundle plan -t dev  
create jobs.ingest
```

```
Plan: 1 to add, 0 to change, 0 to delete, 0 unchanged
```

A create for each resource on the first run; no action for a resource that has not changed. Module 7 runs this same command inside a pull request, where the plan is what a reviewer reads.

Deploying a second time

The same command, with nothing changed

```
$ databricks bundle deploy -t dev  
$ databricks bundle deploy -t dev      # again, no edits
```

Jobs & Pipelines

Jobs & pipelines Runs

[Send feedback](#)

**Ingestion pipeline**
Ingest data from apps, databases and files

**ETL pipeline**
Build ETL pipelines using SQL and Python

**Job**
Orchestrate notebooks, pipelines, queries and more

AllJobsPipelines

Create ▾

Owned by me

Accessible by me

Favorites

Tags ▾

Run as ▾

Name 	Type	Tags	Run as	Trig...	Rece...	
[dev santitham_pro] sales_ingest	 Job	santitham_pro	 SA...	—	—	  

Idempotence is a property of the deployment state under state/, not of the declaration. Delete the state and the second deployment is a first deployment.

bundle run, and what it reports

The run URL first, then the states, then the result

```
$ databricks bundle run ingest -t dev
$ databricks bundle run ingest -t dev --only features
```

```
santitham@data-engineer:~/m4-demo/assets/m4demo$ databricks bundle run ingest -t dev
Run URL: https://adb-1502583690645883.3.azuredatabricks.net/jobs/982723953804016/runs/686370417547577?o=1502583690645883

2026-08-16 15:27:53 "[dev santitham_pro] sales_ingest" RUNNING
2026-08-16 15:29:45 "[dev santitham_pro] sales_ingest" TERMINATED SUCCESS
Output:
=====
Task land:
landed=500 target=main.santitham_pro
=====
Task features:
bands=2 target=main.santitham_pro
=====
Task quality:
checked=500 nulls=0 negative=0
=====
Task publish:
published=main.santitham_pro.sales_daily
=====
Task notify:
notified
santitham@data-engineer:~/m4-demo/assets/m4demo$ databricks bundle run ingest -t dev --only features
Run URL: https://adb-1502583690645883.3.azuredatabricks.net/jobs/982723953804016/runs/469936544772461?o=1502583690645883

2026-08-16 15:30:27 "[dev santitham_pro] sales_ingest" RUNNING
2026-08-16 15:30:58 "[dev santitham_pro] sales_ingest" TERMINATED SUCCESS
```

--only reruns one task of a deployed job without redeploying. Arguments after -- are passed through to the task, which is how you vary a parameter without editing the file.

[Jobs & Pipelines](#) >

 **[dev santitham_pro] sales_ingest**   [Send feedback](#)



Run now



 Connected to Declarative Automation Bundles [Learn more](#)

Disconnect from source

Runs Tasks

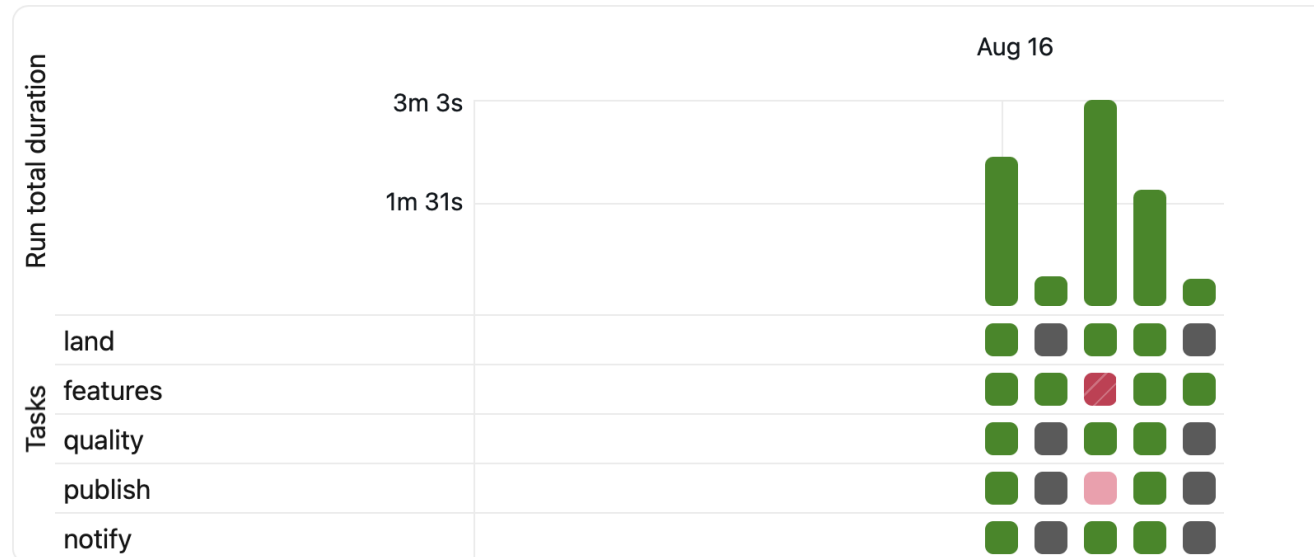
Runs

Started before



< Previous

Next >



Job details



Job ID 982723953804016 

Creator  SANTITHAM PROM-ON

Run as   SANTITHAM PROM-ON

Description [Add description](#)

Lineage  No lineage information for this job.

[Learn more](#) 

Performance optimized Off ☐



Challenge 5 — Work the loop

- 1 Break the file that your second task runs: an unclosed parenthesis is enough.
- 2 Predict, on paper, the state each of your tasks will be reported in. Then deploy and run.
- 3 Read the run history and record the actual state of every task. Note which states your code produced and which the platform produced.
- 4 Fix the file, deploy again, and read what plan reports for the resources that did not change.

Success: A failed task, a downstream task reported separately rather than as a failure, and a second plan that reports no action where nothing changed.

The diagnosis recipe

Four questions, in this order. Each eliminates one class of cause.

- **1. Which command failed: validate, plan, deploy or run?**
This bounds the possible causes before you read the message. A failure at run is not a configuration fault, and a failure at validate is not a workspace fault.
- **2. If deploy: which stage does the message name?**
Build, upload, resource, or state. The stage names the cause: build is your toolchain, upload is permissions or paths, resource is the Jobs API rejecting the definition, state is a conflict with a previous deployment.
- **3. Does the message carry a configuration path and a file position?**
If it does, the fault is in the declaration and the position is the answer. Stop reading and go there.
- **4. If it came from the workspace rather than the CLI, what did we send it?**
Read the field out of bundle validate -o json, not out of the file. The file is not what was sent.

Failure modes and where each is detected

Ordered by the stage that reports them

Symptom	Cause	Detected at
<code>job_cluster_key <k> is not defined</code>	a task claims a cluster nothing declares	validate · warning
<code>unknown field: <key></code>	a misspelled key	validate · warning
<code>invalid value for enum field</code>	a value outside <code>run_if</code> 's six	validate · warning
<code>notebook <path> not found</code>	the path names no file	validate · error
<code>expected a notebook but got a file</code>	the marker line is missing	validate · error
<code>no value assigned to required variable</code>	declared, never assigned, no default	validate · error
<code>a cluster appears that no task claims</code>	a typo in a target override appended	nothing — count them
<code>deploy fails after Validation OK!</code>	an invalid graph, or conflicting compute keys	deploy · resource
<code>a task fails, and those below do not run</code>	the declaration is right; the code is not	run

Challenge 6 — Timed team challenge — a sabotaged bundle

- 1 In teams of three. Copy assets/broken-bundle/ into a scratch directory. It carries four faults, one from each detection stage on the previous slide.
- 2 Work the stages in order. Do not read the whole file first; let the tooling tell you where to look.
- 3 Record the order in which you found them, and which command found each.
- 4 Repair all four, and get the run to a terminal success state.
- 5 Hands up when the run succeeds. The debrief is the order, not the fixes.

Success: validate clean, deploy successful, run successful, and a written order of discovery.

The End of Module 4

If you have any further questions,
please contact me via
santitham.pro@kmutt.ac.th

